

Binary Search Trees

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Binary Search Trees	1
1.1	What Makes a Tree a Search Tree?	1
1.2	Search in a Binary Search Tree	3
1.3	Find the Minimum and Maximum in a BST	5
1.4	Insert a Node in a BST	7
1.5	Delete a Node in a BST	9
1.6	Floor and Ceil in a BST	11
1.7	Kth Smallest Element in a BST	14
1.8	Validate a Binary Search Tree	16
1.9	Predecessor and Successor in a BST	18
1.10	BST Iterator	20
1.11	Two Sum IV: Input is a BST	23
1.12	Lowest Common Ancestor in a BST	25
1.13	Construct a BST from Preorder Traversal	28
1.14	Recover BST (Two Nodes Swapped)	30
1.15	Largest BST in a Binary Tree	33

Chapter 1

Binary Search Trees

A binary search tree is binary search wearing a tree's clothing. Every node carries the same promise an array's sortedness carries – everything in one direction is smaller, everything in the other is larger – and that single promise is enough to make search, insertion, ordered traversal, and rank queries all run in time proportional to the tree's height, not its size.

1.1 What Makes a Tree a Search Tree?

A **binary search tree (BST)** is a binary tree satisfying the **BST invariant**: for every node, every value in its left subtree is strictly less than the node's own value, and every value in its right subtree is strictly greater. This is a direct generalization of the sorted-array invariant that powered the entire Binary Search chapter, restructured so that “go left” or “go right” replaces “halve the search window.”

The One Fact That Powers Almost Every Section in This Chapter

An **inorder traversal** (left subtree, then the node, then right subtree) of a BST always visits nodes in **strictly increasing order**. This is not a coincidence – it is a direct restatement of the BST invariant applied recursively at every node, and it is the single fact that connects BSTs back to everything already proven about sorted arrays in earlier chapters: an inorder traversal of a BST *is*, in effect, a sorted array, just discovered one $O(\log n)$ -depth recursive step at a time instead of stored contiguously.

How to Recognise Which Technique a BST Problem Needs

Recurring BST Keywords

- “**Search/insert/delete a value**” – a single root-to-leaf walk, choosing left or right at each node by comparing to the BST invariant, exactly like binary search choosing which half to discard.
- “**Kth smallest/largest,**” “**floor/ceil,**” or any other *rank*-based query” – either an inorder traversal (which visits values in sorted order for free) or a single guided root-to-leaf walk that narrows toward the answer using the BST invariant directly, without visiting every node.
- “**Validate,**” “**two nodes are swapped,**” or any property phrased in terms of *sortedness*” – think in terms of the inorder sequence being (or not being) sorted, rather than reasoning node-by-node about left/right children directly.
- “**Predecessor/successor,**” or “**iterator that returns values one at a time in sorted order**” – a controlled, space-efficient version of inorder traversal, often using an explicit stack to simulate the recursion only as deep as currently needed.
- “**Largest BST within a general binary tree**” – a bottom-up recursive technique

returning, from each subtree, enough information (is it a valid BST? what is its size? what are its min and max?) for its parent to decide validity and size in $O(1)$ extra work per node, very much in the spirit of dynamic programming on trees.

Each section in this chapter follows the established structure: problem statement and keywords; the reasoning that identifies the right traversal or recursive technique and why it is correct; the algorithm in words and pseudocode; a hand-traced dry run; complete compilable C++ code; and a time/space complexity analysis.

1.2 Search in a Binary Search Tree

Problem Statement. Given the root of a binary search tree and a target value X , find and return the subtree rooted at the node whose value equals X , or return null if no such node exists.

Pattern Recognition

“Search for a value in a BST” is the tree-shaped restatement of ordinary binary search (the Binary Search chapter’s opening section): the BST invariant plays exactly the role sortedness played there, letting a single comparison at each node discard an entire subtree (half the remaining candidates) rather than requiring us to check it at all.

Understanding the problem carefully

At any node, comparing X to that node’s value tells us everything we need: if X equals the node’s value, we are done. If X is smaller, the BST invariant guarantees that *every* value in the right subtree is even larger than the current node, so X cannot possibly be there – discard the right subtree entirely and recurse left. Symmetrically, if X is larger, discard the left subtree and recurse right.

One Comparison, One Subtree Discarded

At each node: if the node is null, X is not present (return null). If X equals the node’s value, return this node. If $X <$ the node’s value, recurse into the left child. If $X >$ the node’s value, recurse into the right child.

Why this is exactly binary search, with depth playing the role of $\log n$ steps

Each step travels one level deeper into the tree and provably eliminates an entire subtree from consideration, using the same discard argument as array-based binary search: the eliminated subtree’s values are *all* on the wrong side of X , guaranteed by the BST invariant, so none of them need to be checked individually. Because each step descends exactly one level, the total number of steps is bounded by the tree’s height – $O(\log n)$ for a balanced tree, though note this degrades to $O(n)$ for a degenerate (linked-list-shaped) tree, a distinction worth remembering throughout this chapter.

Algorithm

1. If the current node is null: return null.
2. If $X =$ current node’s value: return the current node.
3. If $X <$ current node’s value: recurse into the left child.
4. Else: recurse into the right child.

Algorithm 1 Search in a BST

```

1: procedure SEARCHBST(root, X)
2:   if root = null then return null
3:   end if
4:   if X = root.val then return root
5:   end if
6:   if X < root.val then
7:     return SEARCHBST(root.left, X)
8:   else
9:     return SEARCHBST(root.right, X)
10:  end if
11: end procedure

```

Dry run

Dry Run. Take the BST $\begin{matrix} & & 5 & & \\ & 3 & & 8 & \\ 1 & & 4 & & 9 \end{matrix}$ (root 5; left child 3 with children 1, 4; right child 8 with right child 9), searching for $X = 4$.

- At 5: $4 < 5$, go left to 3.
- At 3: $4 > 3$, go right to 4.
- At 4: $4 = 4$, found. Return this node.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11  TreeNode* searchBST(TreeNode* root, int X) {
12      if (root == nullptr || root->val == X) return root;
13      if (X < root->val) return searchBST(root->left, X);
14      return searchBST(root->right, X);
15  }
16
17  int main() {
18      TreeNode* root = new TreeNode(5);
19      root->left = new TreeNode(3);
20      root->right = new TreeNode(8);
21      root->left->left = new TreeNode(1);
22      root->left->right = new TreeNode(4);
23      root->right->right = new TreeNode(9);
24
25      TreeNode* found = searchBST(root, 4);
26      cout << (found ? found->val : -1) << endl;
27      return 0;
28  }

```

Complexity Analysis

Time Complexity. $O(h)$, where h is the tree's height – $O(\log n)$ for a balanced BST, $O(n)$ in the worst case for a degenerate, linked-list-shaped tree. **Space Complexity.** $O(h)$ for the recursion stack (or $O(1)$ if rewritten iteratively, since no backtracking is ever needed).

Key Takeaway

Every BST operation in this chapter that walks from the root toward a single target (search, insert, floor/ceil, predecessor/successor) inherits this exact $O(h)$ complexity and this exact “one comparison, one subtree discarded” justification. Internalizing it once here saves re-deriving it in every later section.

1.3 Find the Minimum and Maximum in a BST

Problem Statement. Given the root of a BST, find its minimum value and its maximum value.

Pattern Recognition

“*Minimum/maximum of a BST*” requires no comparison logic at all, unlike Section 1.2 – the BST invariant alone pins the minimum and maximum to two fixed, unambiguous locations: the **leftmost** node and the **rightmost** node.

Understanding the problem carefully

By the BST invariant, every left child is smaller than its parent, and every right child is larger. So starting from the root and always moving left strictly decreases the value at every step, until no left child remains – at which point we are standing on a value smaller than every other value in the tree (every node we passed is larger, and every node in their right subtrees, which we skipped, is even larger still). The symmetric argument, always moving right, finds the maximum.

Walk to the Edge, No Comparisons Needed

For the minimum: starting at the root, repeatedly move to the left child until there is none; return the value at that point. For the maximum: symmetrically, repeatedly move right until there is none.

Why no value comparisons are needed during the walk

This is a direct structural consequence of the BST invariant, not something that needs separate verification at each step: the invariant *guarantees*, for every node visited, that its left child (if any) is strictly smaller, so simply choosing “left, if it exists” at every step is already provably optimal – there is no need to compare the current node's value to anything, since the tree's shape alone already encodes the answer's location.

Algorithm

1. **FINDMIN:** starting at *root*, while the current node has a left child, move to it. Return the current node's value.
2. **FINDMAX:** symmetrically, while the current node has a right child, move to it. Return the current node's value.

Dry run

Dry Run. Take the BST with root 5, left child 3 (with left child 1 and right child 4), right child 8 (with right child 9).

Min: $5 \rightarrow 3 \rightarrow 1$ (no left child). Minimum is 1.

Max: $5 \rightarrow 8 \rightarrow 9$ (no right child). Maximum is 9.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 int findMin(TreeNode* root) {
12     while (root->left) root = root->left;
13     return root->val;
14 }
15
16 int findMax(TreeNode* root) {
17     while (root->right) root = root->right;
18     return root->val;
19 }
20
21 int main() {
22     TreeNode* root = new TreeNode(5);
23     root->left = new TreeNode(3);
24     root->right = new TreeNode(8);
25     root->left->left = new TreeNode(1);
26     root->left->right = new TreeNode(4);
27     root->right->right = new TreeNode(9);
28
29     cout << "Min: " << findMin(root) << ", Max: " << findMax(root) <<
        endl;
30     return 0;
31 }

```

Complexity Analysis

Time Complexity. $O(h)$, where h is the tree's height. **Space Complexity.** $O(1)$ – the iterative walk needs no recursion stack at all, unlike most other BST operations in this chapter.

Key Takeaway

Finding the minimum and maximum of a BST is the simplest possible operation in this chapter precisely because it needs zero comparisons – the structure alone answers the question. This “walk to the leftmost / rightmost node” primitive is reused directly inside Delete (Section 1.5) to find a replacement node, and inside Predecessor/Successor (Section 1.9).

1.4 Insert a Node in a BST

Problem Statement. Given the root of a BST and a value X to insert, insert X into the tree so that the BST invariant still holds, and return the (possibly new) root.

Pattern Recognition

“Insert into a BST” reuses Section 1.2’s search walk almost unchanged: insertion is what happens when a search for X runs off the edge of the tree (reaches a null pointer) instead of finding a match – that null position is exactly where X belongs.

Understanding the problem carefully

Walk down the tree exactly as in Section 1.2’s search: at each node, go left if X is smaller, right if X is larger. Since X is not yet in the tree, this walk will never find an exact match – it will instead reach a null child pointer. That null position is the unique place where X can be attached while preserving the BST invariant: it is smaller than everything on the path so far that went right, and larger than everything that went left, by construction of the walk itself.

Same Walk as Search, Attach at the Null

Walk from the root as in a search for X . Whenever the walk would move into a null child, create a new node with value X and attach it there instead. If the tree was empty to begin with, the new node simply becomes the root.

Why attaching at the first null reached is always valid

Every step of the walk maintains an invariant: X is consistent with becoming a descendant of the current node, given everything decided so far (it is greater than every ancestor we went right from, and less than every ancestor we went left from). Reaching a null pointer means no existing node currently occupies the position consistent with all those decisions – so creating a new node there extends the tree without violating the BST invariant anywhere, since the new node’s relationship to every ancestor on the path is already exactly correct by construction.

Algorithm

1. If the current node is null: create and return a new node with value X .
2. If $X <$ current node’s value: recursively insert into the left child, and attach the (possibly updated) result back as the left child.
3. Else: recursively insert into the right child, attach back as the right child.
4. Return the current node (unchanged, except possibly for the newly attached child).

Algorithm 2 Insert into a BST

```

1: procedure INSERTBST(root, X)
2:   if root = null then
3:     return new node with value X
4:   end if
5:   if X < root.val then
6:     root.left ← INSERTBST(root.left, X)
7:   else
8:     root.right ← INSERTBST(root.right, X)
9:   end if
10:  return root
11: end procedure

```

Dry run

Dry Run. Insert $X = 6$ into the BST with root 5, left child 3, right child 8.

- At 5: $6 > 5$, go right to 8.
- At 8: $6 < 8$, go left – but 8 has no left child (null). Create new node 6, attach as 8's left child.

The tree now has 8's left child equal to 6.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11  TreeNode* insertBST(TreeNode* root, int X) {
12      if (root == nullptr) return new TreeNode(X);
13
14      if (X < root->val) root->left = insertBST(root->left, X);
15      else root->right = insertBST(root->right, X);
16
17      return root;
18  }
19
20  int main() {
21      TreeNode* root = new TreeNode(5);
22      root->left = new TreeNode(3);
23      root->right = new TreeNode(8);
24
25      insertBST(root, 6);
26      cout << "8's left child: " << root->right->left->val << endl;
27      return 0;
28  }

```

Complexity Analysis

Time Complexity. $O(h)$, identical to search. **Space Complexity.** $O(h)$ for the recursion stack, plus $O(1)$ for the single new node created.

Key Takeaway

Insertion is search that creates a node instead of reporting failure – recognizing this means you do not need to separately re-derive the walk logic, only add the “attach a new node here” step at the point where search would otherwise give up.

1.5 Delete a Node in a BST

Problem Statement. Given the root of a BST and a value X , delete the node with value X (if present) and return the root of the resulting BST, which must still satisfy the BST invariant.

Pattern Recognition

“Delete from a BST” is the hardest of the three basic operations because, unlike search or insert, the node being removed might be in the **middle** of the tree with two children that both need to stay connected to the rest of the tree somehow. The keyword to notice: deletion needs a **replacement value** that preserves the BST invariant, and Section 1.3’s “walk to the edge” primitive is exactly what finds one.

Understanding the problem carefully

First, locate the node with value X using the same walk as Section 1.2. Once found, there are three structurally different cases:

- **Leaf node (no children).** Simply remove it – nothing else needs to change.
- **One child.** Replace the node with its single child – the parent’s pointer to this node is redirected to point at the child instead, and the BST invariant is automatically preserved, since that child’s entire subtree already sat correctly on one side of the now-removed node.
- **Two children.** This is the hard case: we cannot just remove the node, since that would disconnect two subtrees from each other. Instead, find a **replacement value** that can legally take this node’s place – the smallest value in the right subtree (its inorder successor) or, symmetrically, the largest value in the left subtree (its inorder predecessor) both work. Copy that replacement value into the current node, then recursively delete the replacement’s *original* location from whichever subtree it came from (which, being an edge value of that subtree, will only ever fall into the leaf or one-child case, never back into the two-children case).

Two-Children Case: Borrow the Successor, Then Delete It Recursively

When deleting a node with two children, find its inorder successor (the minimum of its right subtree, found via Section 1.3’s walk), copy that successor’s value into the current node, and then recursively delete that same value from the right subtree – which now contains a duplicate, but only temporarily, and the recursive delete call removes the original occurrence cleanly.

Why borrowing the successor's value, then deleting it, is always correct

The inorder successor of a two-children node is, by definition, the smallest value strictly greater than the node's current value – which means it is simultaneously larger than everything in the node's left subtree (since it's larger than the node itself) and smaller than or equal to everything else remaining in the right subtree (since it's the minimum of that subtree). So placing it directly into the current node's position preserves the BST invariant relative to both subtrees perfectly. And because the successor is the minimum of the right subtree, it has no left child (a node with a left child cannot be a subtree's minimum), so deleting its original location is guaranteed to fall into the simple leaf-or-one-child case, never recursing into another two-children scenario at that step.

Algorithm

1. If the current node is null: return null (X was not found; nothing to delete).
2. If $X <$ current node's value: recurse left, reattach the result.
3. If $X >$ current node's value: recurse right, reattach the result.
4. Else (found the node to delete):
 - (a) If it has no left child: return its right child (handles both the leaf case and the one-right-child case).
 - (b) If it has no right child: return its left child.
 - (c) Otherwise (two children): find *successor* $\leftarrow$ the minimum value in the right subtree. Set the current node's value to *successor*. Recursively delete *successor* from the right subtree, and reattach the result as the new right child.
5. Return the (possibly updated) current node.

Dry run

Dry Run. Delete $X = 5$ from the BST with root 5, left child 3, right child 8 (with left child 7 and right child 9).

At the root ($5 = X$): two children present. Find successor: minimum of right subtree (rooted at 8) is reached by walking left: $8 \rightarrow 7$ (no left child) – successor is 7. Set root's value to 7. Recursively delete 7 from the right subtree (rooted at 8): at 8, $7 < 8$, go left to 7; at 7 ($7 = X$ for this recursive call), no children – return null, attach as 8's new left child.

Resulting tree: root 7, left child 3, right child 8 (now with no left child, right child 9).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 int findMin(TreeNode* root) {
12     while (root->left) root = root->left;

```

```

13     return root->val;
14 }
15
16 TreeNode* deleteNode(TreeNode* root, int X) {
17     if (root == nullptr) return nullptr;
18
19     if (X < root->val) {
20         root->left = deleteNode(root->left, X);
21     } else if (X > root->val) {
22         root->right = deleteNode(root->right, X);
23     } else {
24         if (root->left == nullptr) return root->right;
25         if (root->right == nullptr) return root->left;
26
27         int successor = findMin(root->right);
28         root->val = successor;
29         root->right = deleteNode(root->right, successor);
30     }
31     return root;
32 }
33
34 int main() {
35     TreeNode* root = new TreeNode(5);
36     root->left = new TreeNode(3);
37     root->right = new TreeNode(8);
38     root->right->left = new TreeNode(7);
39     root->right->right = new TreeNode(9);
40
41     root = deleteNode(root, 5);
42     cout << "New root: " << root->val << endl;
43     return 0;
44 }

```

Complexity Analysis

Time Complexity. $O(h)$: at most one $O(h)$ walk to find the node, plus at most one more $O(h)$ walk to find and delete the successor in the two-children case. **Space Complexity.** $O(h)$ for the recursion stack.

Key Takeaway

The hard case of BST deletion is always solved the same way: borrow a boundary value (successor or predecessor) from the subtree that must stay connected, copy it up, then delete the boundary value's original location – which is guaranteed to be an easy case by construction. This “borrow an edge value, then recursively clean up” pattern is worth remembering as its own primitive, independent of the BST context.

1.6 Floor and Ceil in a BST

Problem Statement. Given the root of a BST and a value X , find the **floor** of X (the largest value in the tree that is $\leq X$) and the **ceil** of X (the smallest value in the tree that is $\geq X$).

Pattern Recognition

This is the tree-shaped version of the Binary Search chapter's Floor and Ceil in a Sorted Array problem: the same “record a tentative answer, keep narrowing toward the boundary” template applies, except narrowing now means moving to a child rather than shrinking an array window.

Understanding the problem carefully

Walk from the root, comparing X to each node's value. If the current node's value is exactly X , it is simultaneously its own floor and ceil – done immediately. If the current node's value is less than X , it is a *candidate* floor (everything in its left subtree is even smaller, so it cannot help find a *larger* floor candidate, but the right subtree might contain an even closer one) – record it and move right. Symmetrically, if the current node's value exceeds X , it is a candidate ceil – record it and move left.

Record a Candidate, Narrow Toward a Closer One

Maintain $floor \leftarrow -\infty$ (or undefined) and $ceil \leftarrow +\infty$ (or undefined). At each node: if its value equals X , both floor and ceil are this value – stop. If its value $< X$: update $floor$ to this value (a closer candidate than any previous one, since we are moving rightward, deeper toward X), and move right. If its value $> X$: update $ceil$, and move left.

Why moving right after finding a smaller value always finds a closer floor (if one exists)

Once we know the current node's value is $< X$ and is therefore a valid floor candidate, any *better* (larger, but still $\leq X$) candidate can only exist in the current node's right subtree – the BST invariant guarantees every value in the right subtree exceeds the current node's value, narrowing the search toward X from below, while the left subtree only contains values even smaller, which could never improve on the current candidate. This mirrors exactly the boundary-narrowing logic of the Binary Search chapter's array-based floor/ceil, with “move right in the tree” playing the role that “move the binary search window toward the boundary” played there.

Algorithm

1. Initialize $floor, ceil \leftarrow$ undefined.
2. While the current node is not null:
 - (a) If current value = X : set both $floor$ and $ceil$ to X ; stop.
 - (b) If current value $< X$: $floor \leftarrow$ current value; move to the right child.
 - (c) Else: $ceil \leftarrow$ current value; move to the left child.
3. Return $(floor, ceil)$.

Dry run

Dry Run. Take the BST with root 8, left subtree rooted at 3 (children 1 and 6, where 6 has left child 4), right subtree rooted at 10 (left child 9, right child 14). Query $X = 7$.

- At 8: $8 > 7$, $ceil = 8$. Move left to 3.
- At 3: $3 < 7$, $floor = 3$. Move right to 6.

- At 6: $6 < 7$, $\text{floor} = 6$. Move right – but 6 has no right child (null). Stop.

Result: $\text{floor} = 6$, $\text{ceil} = 8$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 pair<int,int> floorCeilBST(TreeNode* root, int X) {
12     int floorVal = -1, ceilVal = -1;
13     TreeNode* node = root;
14
15     while (node) {
16         if (node->val == X) {
17             floorVal = ceilVal = X;
18             break;
19         } else if (node->val < X) {
20             floorVal = node->val;
21             node = node->right;
22         } else {
23             ceilVal = node->val;
24             node = node->left;
25         }
26     }
27     return {floorVal, ceilVal};
28 }
29
30 int main() {
31     TreeNode* root = new TreeNode(8);
32     root->left = new TreeNode(3);
33     root->left->left = new TreeNode(1);
34     root->left->right = new TreeNode(6);
35     root->left->right->left = new TreeNode(4);
36     root->right = new TreeNode(10);
37     root->right->left = new TreeNode(9);
38     root->right->right = new TreeNode(14);
39
40     auto [floorVal, ceilVal] = floorCeilBST(root, 7);
41     cout << "Floor: " << floorVal << ", Ceil: " << ceilVal << endl;
42     return 0;
43 }

```

Complexity Analysis

Time Complexity. $O(h)$, a single root-to-(leaf or match) walk. **Space Complexity.** $O(1)$ for the iterative version shown (no recursion needed, since neither subtree is ever revisited).

Key Takeaway

Floor and ceil in a BST need only a single $O(h)$ walk, never a full traversal – a useful contrast with Kth Smallest (Section 1.7), where, without extra bookkeeping, an inorder traversal touching many nodes is genuinely required. Whenever a BST query can be answered by a sequence of single left/right decisions without needing to inspect both children’s full subtrees, prefer the guided walk over a full traversal.

1.7 Kth Smallest Element in a BST

Problem Statement. Given the root of a BST and an integer k , find the k -th smallest value stored in the tree.

Pattern Recognition

“*Kth smallest*” on a BST is answered directly by Section 1.1’s headline fact: an **inorder traversal visits values in strictly increasing order**. So the k -th value produced by an inorder traversal *is* the k -th smallest value in the tree – no sorting, and no need to visit the whole tree if we stop as soon as the k -th value is produced.

Understanding the problem carefully

A recursive inorder traversal could be used, but stopping it early (the moment the k -th value is produced) is awkward with plain recursion, since recursive calls don’t have a natural way to “stop everything” once we are several levels deep. An **iterative** inorder traversal, using an explicit stack to simulate the recursion, gives full control over exactly when to stop – we simply count nodes as we pop them and halt the moment the count reaches k .

Iterative Inorder Traversal, Counting as We Go

Maintain an explicit stack. Repeatedly: push the current node and move left, until reaching null (this pushes the entire leftward chain, since inorder visits left subtrees first). Then pop a node – this is the next value in sorted order; increment a counter, and if the counter reaches k , return this node’s value immediately. Otherwise, move to the popped node’s right child and repeat the whole process (push its leftward chain, then continue popping).

Why the iterative version visits nodes in exactly inorder sequence

Pushing the entire leftward chain before popping anything ensures that, when we do pop, we are popping the smallest value not yet visited: any node still left to visit that is smaller than the current pop candidate would have to be in some left subtree along the path already pushed, and all of those are already on the stack, ready to be popped first (stacks pop most-recently-pushed first, which corresponds to the deepest, leftmost, smallest unvisited node). After popping a node, moving to its right child and repeating correctly resumes the traversal exactly where recursive inorder would have continued, since everything in that node’s own left subtree has already been fully visited.

Algorithm

1. Initialize an empty stack, $count \leftarrow 0$, $node \leftarrow root$.
2. While $node$ is not null or the stack is not empty:

- (a) While *node* is not null: push it; $node \leftarrow node.left$.
- (b) Pop *node*; $count \leftarrow count + 1$.
- (c) If $count = k$: return $node.val$.
- (d) $node \leftarrow node.right$.

Dry run

Dry Run. Take the BST with root 5, left child 3 (children 1, 4), right child 8, $k = 3$.

- Push leftward chain from 5: push 5, push 3, push 1 (no left child). Stack (top to bottom) = [1, 3, 5].
- Pop 1: $count = 1 \neq 3$. Move to 1's right (null).
- Push leftward chain from null: nothing to push.
- Pop 3: $count = 2 \neq 3$. Move to 3's right, which is 4.
- Push leftward chain from 4: push 4 (no left child).
- Pop 4: $count = 3 = k$. Return 4.

The 3rd smallest value is 4 (inorder sequence: 1, 3, 4, 5, 8).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 int kthSmallest(TreeNode* root, int k) {
12     stack<TreeNode*> st;
13     TreeNode* node = root;
14     int count = 0;
15
16     while (node || !st.empty()) {
17         while (node) {
18             st.push(node);
19             node = node->left;
20         }
21         node = st.top();
22         st.pop();
23         count++;
24         if (count == k) return node->val;
25         node = node->right;
26     }
27     return -1; // k larger than tree size
28 }
29
30 int main() {

```

```

31  TreeNode* root = new TreeNode(5);
32  root->left = new TreeNode(3);
33  root->right = new TreeNode(8);
34  root->left->left = new TreeNode(1);
35  root->left->right = new TreeNode(4);
36
37  cout << "3rd smallest: " << kthSmallest(root, 3) << endl;
38  return 0;
39 }

```

Complexity Analysis

Time Complexity. $O(h + k)$: pushing the initial leftward chain costs $O(h)$, and each subsequent step processes one more node until the k -th is found, giving $O(h + k)$ overall – in the worst case $O(n)$ if k is close to the total node count. **Space Complexity.** $O(h)$ for the stack.

Key Takeaway

Whenever a BST problem needs the i -th value in sorted order, or values processed one at a time in sorted order with the ability to stop early, an *iterative* inorder traversal with an explicit stack is the right tool – it gives the same sorted-order guarantee as recursive inorder, with full control over when to stop. This exact iterative inorder machinery, generalized to resume across multiple separate calls, is what Section 1.10's BST Iterator formalizes next.

1.8 Validate a Binary Search Tree

Problem Statement. Given the root of a binary tree, determine whether it is a valid BST: for every node, all values in its left subtree must be strictly less than the node's value, and all values in its right subtree must be strictly greater.

Pattern Recognition

The keyword trap here: it is tempting to check only that each node is greater than its immediate left child and less than its immediate right child, but the BST invariant is actually about **every** node in each subtree, not just the direct children. “*Validate a property that must hold against **all** ancestors, not just the immediate parent*” signals that the recursion needs to carry a **valid range** downward, narrowing at every step, rather than just comparing parent to child.

Understanding the problem carefully

Consider the tree with root 5, left child 3, and 3's right child 6. Checking only parent-child pairs would see: $3 < 5$ (fine, left child smaller than parent) and $6 > 3$ (fine, right child larger than its own parent) – and incorrectly conclude this is a valid BST. But 6 sits in 5's *left* subtree, and the BST invariant requires *every* value there to be less than 5, which 6 violates. The bug is checking only local parent-child relationships instead of the full inherited range of validity.

Pass Down a Valid (Lower, Upper) Range

Recurse with two bounds, *lower* and *upper* (initially $-\infty$ and $+\infty$ at the root). At each node: if its value is not strictly between *lower* and *upper*, the tree is invalid. Otherwise, recurse left with the same *lower* but *upper* tightened to the current node's value (everything in the left subtree must be less than this node), and recurse right with *lower* tightened to

the current node's value and the same *upper*.

Why narrowing the range at every level correctly enforces the global invariant

By construction, the range (*lower*, *upper*) passed into any node already reflects the combined constraints of *every* ancestor on the path from the root, not just the immediate parent: each recursive call only ever tightens the range, never loosens it, so a deep descendant's range is the *intersection* of every constraint imposed by every ancestor above it. Checking the current node's value against this fully-accumulated range is therefore equivalent to checking it against every single ancestor directly, just done incrementally and in $O(1)$ work per node instead of re-walking ancestors at every step.

Algorithm

1. `VALIDATE(node, lower, upper):`
 - (a) If *node* = null: return true (an empty tree is trivially valid).
 - (b) If *node.val* ≤ *lower* or *node.val* ≥ *upper*: return false.
 - (c) Return `VALIDATE(node.left, lower, node.val)` **and** `VALIDATE(node.right, node.val, upper)`.
2. Call with *lower* = $-\infty$, *upper* = $+\infty$ at the root.

Dry run

Dry Run. Take the tree with root 5, left child 3, 3's right child 6.

- At 5, range $(-\infty, +\infty)$: 5 is within range. Recurse left into 3 with range $(-\infty, 5)$; recurse right into null with range $(5, +\infty)$ (trivially valid).
- At 3, range $(-\infty, 5)$: 3 is within range. Recurse left into null (valid); recurse right into 6 with range $(3, 5)$.
- At 6, range $(3, 5)$: is 6 strictly between 3 and 5? No ($6 \geq 5$). Return false.

The tree is correctly identified as **invalid**.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 bool validate(TreeNode* node, long long lower, long long upper) {
12     if (node == nullptr) return true;
13     if (node->val <= lower || node->val >= upper) return false;
14
15     return validate(node->left, lower, node->val) &&
16         validate(node->right, node->val, upper);

```

```

17 }
18
19 bool isValidBST(TreeNode* root) {
20     return validate(root, LLONG_MIN, LLONG_MAX);
21 }
22
23 int main() {
24     TreeNode* root = new TreeNode(5);
25     root->left = new TreeNode(3);
26     root->left->right = new TreeNode(6);
27
28     cout << (isValidBST(root) ? "true" : "false") << endl;
29     return 0;
30 }

```

Complexity Analysis

Time Complexity. Every node is visited exactly once, with $O(1)$ work per node, giving $O(n)$.

Space Complexity. $O(h)$ for the recursion stack.

Key Takeaway

Any property phrased as “true for every node relative to *all* of its ancestors/descendants,” not just its immediate neighbors, needs state (here, a valid range) threaded through the recursion, accumulating constraints as it descends. An equally valid alternative solution performs an inorder traversal and checks that the resulting sequence is strictly increasing, directly invoking Section 1.1’s headline fact – worth knowing as a second way to arrive at the same answer.

1.9 Predecessor and Successor in a BST

Problem Statement. Given the root of a BST and a key X (which may or may not itself be present in the tree), find the **inorder predecessor** of X (the largest value in the tree strictly less than X) and the **inorder successor** of X (the smallest value strictly greater than X).

Pattern Recognition

Predecessor and successor are floor and ceil (Section 1.6) with **strict** inequalities instead of non-strict ones – the keyword difference between “floor” (largest $\leq X$) and “predecessor” (largest $< X$) is exactly the difference between $\geq$ and $>$ that distinguished lower bound from upper bound back in the Binary Search chapter.

Understanding the problem carefully

The walk is identical in shape to floor/ceil: at each node, compare to X and decide which candidate to update and which direction to move. The only change is that when the current node’s value equals X *exactly*, we must **not** stop immediately (unlike floor/ceil, where an exact match ends the search for both) – a predecessor or successor is required to be *strictly* different from X , so if the current node’s value equals X , we must continue descending: moving left to find a predecessor strictly smaller, or right to find a successor strictly larger.

Same Walk as Floor/Ceil, but Treat an Exact Match as “Keep Going”

For the predecessor: at each node, if its value $< X$ (this includes the case of looking for a value strictly less, so equality to X does *not* qualify): record it as the current best predecessor candidate, and move right (search for an even closer one). If its value $\geq X$: do not record anything; move left. Symmetrically for the successor, with roles of $</\geq$ swapped to $>/\leq$.

Why treating equality as “move past it” (not “stop”) is the only change needed

In floor/ceil, an exact match is itself simultaneously the floor and the ceil, so the search correctly terminates there. In predecessor/successor, an exact match is explicitly excluded from being its own answer, so the search must instead behave exactly as if that node were just slightly too small (for predecessor purposes, treat it like any other “ $\geq X$ ” node and move left without recording it) or slightly too large (for successor purposes, move right without recording it) – everything else about the boundary-narrowing logic carries over unchanged from Section 1.6.

Algorithm

1. PREDECESSOR: $pred \leftarrow \text{undefined}$, $node \leftarrow \text{root}$. While $node$ is not null: if $node.val < X$: $pred \leftarrow node.val$; move right. Else: move left.
2. SUCCESSOR: symmetric, with $>$ in place of $<$ and left/right swapped.

Dry run

Dry Run. Take the BST with root 8, left child 3 (children 1, 6, where 6 has left child 4), right child 10 (children 9, 14). Query $X = 6$.

Predecessor: At 8: $8 < 6$? No, move left to 3. At 3: $3 < 6$? Yes, $pred = 3$, move right to 6. At 6: $6 < 6$? No (equal, not strictly less), move left to 4. At 4: $4 < 6$? Yes, $pred = 4$, move right (null). Stop. Predecessor = 4.

Successor: At 8: $8 > 6$? Yes, $succ = 8$, move left to 3. At 3: $3 > 6$? No, move right to 6. At 6: $6 > 6$? No (equal), move right (null). Stop. Successor = 8.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 int findPredecessor(TreeNode* root, int X) {
12     int pred = -1;
13     while (root) {
14         if (root->val < X) {
15             pred = root->val;
16             root = root->right;
17         } else {
18             root = root->left;

```

```

19     }
20 }
21 return pred;
22 }
23
24 int findSuccessor(TreeNode* root, int X) {
25     int succ = -1;
26     while (root) {
27         if (root->val > X) {
28             succ = root->val;
29             root = root->left;
30         } else {
31             root = root->right;
32         }
33     }
34     return succ;
35 }
36
37 int main() {
38     TreeNode* root = new TreeNode(8);
39     root->left = new TreeNode(3);
40     root->left->left = new TreeNode(1);
41     root->left->right = new TreeNode(6);
42     root->left->right->left = new TreeNode(4);
43     root->right = new TreeNode(10);
44     root->right->left = new TreeNode(9);
45     root->right->right = new TreeNode(14);
46
47     cout << "Predecessor: " << findPredecessor(root, 6) << endl;
48     cout << "Successor: " << findSuccessor(root, 6) << endl;
49     return 0;
50 }

```

Complexity Analysis

Time Complexity. $O(h)$ for each of predecessor and successor. **Space Complexity.** $O(1)$ for the iterative version shown.

Key Takeaway

Predecessor/successor and floor/ceil are the same boundary-search template; the only behavioral difference is what happens on an exact match (stop immediately for floor/ceil, keep narrowing for predecessor/successor). When given a problem statement, check carefully whether equality should count as a valid answer or must be strictly excluded – that single detail is the entire distinction between these two near-identical problems.

1.10 BST Iterator

Problem Statement. Design an iterator over a BST that supports *next()* (returns the next smallest value in the tree, in sorted order) and *hasNext()* (whether any values remain), initialized with the tree's root. Across the iterator's whole lifetime, *next* should average $O(1)$ time and the iterator should use $O(h)$ space, not $O(n)$.

Pattern Recognition

“Iterator returning sorted values one at a time” is exactly Section 1.7’s iterative inorder traversal, **paused and resumed** across separate method calls instead of run to completion inside a single function. The keyword shift from “compute the k -th value” to “design an iterator” is a signal that the same stack-based traversal state needs to persist as object state between calls, rather than living entirely within one function’s local variables.

Understanding the problem carefully

If we simply ran a full inorder traversal up front and stored all n values in a list, $next()$ would be trivially $O(1)$, but construction would cost $O(n)$ time *and* space – failing the $O(h)$ space requirement. The fix is to keep *only* the explicit stack from Section 1.7’s iterative traversal alive as the iterator’s state, advancing it by exactly one step per $next()$ call instead of running it to completion all at once.

Persist the Traversal Stack as Iterator State

On construction: push the entire leftward chain from the root onto the stack (exactly as in Section 1.7’s setup). $hasNext()$: simply check whether the stack is non-empty. $next()$: pop the top of the stack (this is the next value in sorted order); then push the leftward chain starting from the popped node’s right child (preparing the stack so that the *following* call to $next$ will correctly find the next value after this one); return the popped value.

Why pushing only the new leftward chain after each pop keeps amortized cost low

Each node in the tree is pushed onto the stack exactly once during the iterator’s entire lifetime (either during construction, for nodes on the initial leftward chain, or during some later $next()$ call, when it becomes the leftward-chain push triggered by its parent being popped), and each node is popped exactly once. So the *total* work across all calls to $next()$ over the iterator’s lifetime is $O(n)$ (each node contributes $O(1)$ amortized push/pop work) – which, spread over the n calls needed to exhaust the tree, gives $O(1)$ *average* time per call, even though any single call might cost up to $O(h)$ if it happens to push a long leftward chain.

Algorithm

1. $CONSTRUCTOR(root)$: push the leftward chain from $root$ onto the stack.
2. $HASNEXT()$: return whether the stack is non-empty.
3. $NEXT()$: pop $node$ from the stack; push the leftward chain starting from $node.right$; return $node.val$.

Dry run

Dry Run. Take the BST with root 7, left child 3, right child 15 (left child 9, right child 20). Call $next$ three times.

- Construct: push leftward chain from 7: push 7, push 3 (no left child). Stack (top to bottom) = $[3, 7]$.
- $next()$: pop 3. Push leftward chain from 3’s right (null) – nothing to push. Stack = $[7]$. Return 3.

- *next()*: pop 7. Push leftward chain from 7's right, 15: push 15, then push 15's left, 9 (no left child). Stack = [9, 15]. Return 7.
- *next()*: pop 9. Push leftward chain from 9's right (null) – nothing. Stack = [15]. Return 9.

Output sequence so far: 3, 7, 9 – correctly sorted order.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 class BSTIterator {
12     stack<TreeNode*> st;
13
14     void pushLeftChain(TreeNode* node) {
15         while (node) {
16             st.push(node);
17             node = node->left;
18         }
19     }
20
21 public:
22     BSTIterator(TreeNode* root) {
23         pushLeftChain(root);
24     }
25
26     bool hasNext() {
27         return !st.empty();
28     }
29
30     int next() {
31         TreeNode* node = st.top();
32         st.pop();
33         pushLeftChain(node->right);
34         return node->val;
35     }
36 };
37
38 int main() {
39     TreeNode* root = new TreeNode(7);
40     root->left = new TreeNode(3);
41     root->right = new TreeNode(15);
42     root->right->left = new TreeNode(9);
43     root->right->right = new TreeNode(20);
44
45     BSTIterator it(root);
46     while (it.hasNext()) cout << it.next() << " ";
47     cout << endl;
48     return 0;
49 }

```

Complexity Analysis

Time Complexity. $O(1)$ average per *next()* call (amortized over the iterator’s lifetime), $O(h)$ worst case for any single call; *hasNext()* is always $O(1)$. **Space Complexity.** $O(h)$ for the stack, never $O(n)$.

Key Takeaway

Converting a one-shot traversal into a resumable iterator is purely a matter of moving the traversal’s local state (here, the explicit stack) into object-level state that persists between calls. The traversal logic itself – push the leftward chain, pop, push the popped node’s right subtree’s leftward chain – is completely unchanged from Section 1.7.

1.11 Two Sum IV: Input is a BST

Problem Statement. Given the root of a BST and a target integer k , determine whether there exist two **distinct** nodes whose values sum to exactly k .

Pattern Recognition

“Two sum” on a BST is the classic two-pointer two-sum technique (normally run on a sorted array) combined with Section 1.10’s BST Iterator: since an inorder traversal already produces values in sorted order on demand, we can drive **two** iterators – one walking forward (ascending) from the smallest value, one walking backward (descending) from the largest – exactly as the classic two-pointer two-sum algorithm drives two pointers inward from the two ends of a sorted array, without ever materializing that sorted array.

Understanding the problem carefully

If the tree’s values were laid out as a sorted array, the standard two-sum technique would maintain a pointer at each end: compute the sum of the two pointed-at values; if it equals k , done; if it is too small, advance the left pointer rightward (to a larger value); if too large, move the right pointer leftward (to a smaller value). A BST supports exactly this access pattern *without* an explicit array, using two independent, resumable inorder-traversal iterators: one producing ascending values on demand (Section 1.10’s iterator exactly as built), and a mirror-image iterator producing descending values on demand (the same idea, with left/right swapped).

Two BST Iterators, Driven Like Two-Sum’s Two Pointers

Initialize a forward iterator *lo* (smallest-first) and a backward iterator *hi* (largest-first), both via the stack-based technique from Section 1.10. Maintain their current values $a = lo.next()$ and $b = hi.next()$. While $a < b$: if $a + b = k$, return true; if $a + b < k$, advance *lo* (get a larger a); if $a + b > k$, advance *hi* (get a smaller b). If the pointers meet or cross ($a \geq b$) without finding a match, return false.

Why driving two independent stack-based iterators is correct and efficient

This is exactly the two-pointer two-sum algorithm, with the “array” it normally operates on replaced by two on-demand sorted streams generated directly from the tree. Each iterator independently maintains its own $O(h)$ -space stack, advancing forward or backward through the BST’s sorted order exactly as Section 1.10 established, and because both iterators only ever

move *toward* each other (never revisiting a value), the total work across the entire search is bounded by the number of *distinct* values each iterator visits before they cross – giving the same linear-total-work guarantee as the classic array-based two-pointer technique, without ever needing $O(n)$ space to store a flattened sorted array.

Algorithm

1. Initialize a forward BST iterator lo and a backward BST iterator hi .
2. $a \leftarrow lo.next()$, $b \leftarrow hi.next()$.
3. While $a < b$:
 - (a) If $a + b = k$: return true.
 - (b) If $a + b < k$: $a \leftarrow lo.next()$.
 - (c) Else: $b \leftarrow hi.next()$.
4. Return false.

Dry run

Dry Run. Take the BST with root 5, left child 3, right child 8, target $k = 10$.

Forward iterator yields 3, 5, 8 in order; backward iterator yields 8, 5, 3 in order.

- $a = lo.next() = 3$, $b = hi.next() = 8$. $a < b$. $a + b = 11 > 10$. Advance hi : $b = 5$.
- $a = 3 < b = 5$. $a + b = 8 < 10$. Advance lo : $a = 5$.
- $a = 5$, $b = 5$: $a < b$ is now false ($5 \not< 5$). Loop ends.

Return false. Checking by hand: the only pair sums available are $3+5=8$, $3+8=11$, $5+8=13$ – none equal 10. Correct.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 class BSTIterator {
12     stack<TreeNode*> st;
13     bool reverseOrder; // true = descending (successor uses right first)
14
15     void pushChain(TreeNode* node) {
16         while (node) {
17             st.push(node);
18             node = reverseOrder ? node->right : node->left;
19         }
20     }

```

```

21
22 public:
23     BSTIterator(TreeNode* root, bool reverse) : reverseOrder(reverse) {
24         pushChain(root);
25     }
26
27     int next() {
28         TreeNode* node = st.top();
29         st.pop();
30         pushChain(reverseOrder ? node->left : node->right);
31         return node->val;
32     }
33 };
34
35 bool findTarget(TreeNode* root, int k) {
36     BSTIterator lo(root, false); // ascending
37     BSTIterator hi(root, true);  // descending
38
39     int a = lo.next(), b = hi.next();
40     while (a < b) {
41         if (a + b == k) return true;
42         else if (a + b < k) a = lo.next();
43         else b = hi.next();
44     }
45     return false;
46 }
47
48 int main() {
49     TreeNode* root = new TreeNode(5);
50     root->left = new TreeNode(3);
51     root->right = new TreeNode(8);
52
53     cout << (findTarget(root, 10) ? "true" : "false") << endl;
54     cout << (findTarget(root, 11) ? "true" : "false") << endl;
55     return 0;
56 }

```

Complexity Analysis

Time Complexity. $O(n)$ in the worst case – each value is produced by one of the two iterators at most once before they cross, and each iterator's amortized cost per call is $O(1)$, as established in Section 1.10. **Space Complexity.** $O(h)$ for each iterator's stack, so $O(h)$ total.

Key Takeaway

A BST is, for the purposes of any algorithm that only needs sorted-order access, a drop-in replacement for a sorted array – as long as you have an $O(h)$ -space, resumable iterator (forward and/or backward) instead of an $O(n)$ -space materialized array. Any two-pointer array technique (two-sum, partitioning, merging) can be ported to a BST this way, exactly as done here.

1.12 Lowest Common Ancestor in a BST

Problem Statement. Given the root of a BST and two nodes p and q (both guaranteed to exist in the tree), find their **lowest common ancestor** (LCA): the deepest node that has both

p and q as descendants (a node is considered a descendant of itself).

Pattern Recognition

LCA in a general binary tree requires examining both subtrees at every node and combining results bottom-up, costing $O(n)$. The keyword “*BST*” attached to an LCA problem is a strong signal that the BST invariant alone determines the split point directly, top-down, in $O(h)$ – no need to inspect both subtrees blindly the way a general tree’s LCA algorithm would.

Understanding the problem carefully

In a BST, the LCA of p and q is exactly the node where the search paths to p and q **diverge**: as long as both $p.val$ and $q.val$ are on the same side (both less than the current node, or both greater), the current node cannot be the LCA – both targets are reachable only by continuing in that single, same direction, so the LCA must lie further down on that side. The moment p and q fall on *different* sides (or one of them *is* the current node), the current node is exactly the point where their paths separate – the LCA.

Follow Both Targets Together Until They Diverge

Starting at the root: if both $p.val$ and $q.val$ are less than the current node’s value, move left. If both are greater, move right. Otherwise (one is $\leq$ and the other is $\geq$ the current node’s value, including the case where the current node *is* p or q), the current node is the LCA.

Why the divergence point is provably the LCA

If both targets are smaller than the current node, the current node cannot be an ancestor that is also the *lowest* one – both p and q are confirmed to live in the left subtree, so a deeper common ancestor must exist there, and continuing left can only get closer to it (never skip past it, since the BST invariant guarantees the entire left subtree is exactly where both targets are). The symmetric argument holds for both being larger. The first node where this is no longer true – where the two targets are no longer both on the same side – is, by definition, the deepest point both search paths still share, since any node below it would have to continue toward only one of p or q , not both.

Algorithm

1. Start at the root.
2. While true:
 - (a) If $p.val < current.val$ and $q.val < current.val$: move to the left child.
 - (b) Else if $p.val > current.val$ and $q.val > current.val$: move to the right child.
 - (c) Else: return *current* (this is the LCA).

Dry run

Dry Run. Take the BST with root 6, left child 2 (children 0, 4), right child 8 (children 7, 9). Find LCA of $p = 2$, $q = 4$.

- At 6: is $2 < 6$ and $4 < 6$? Yes, both smaller. Move left to 2.

- At 2: is $2 < 2$? No (p equals the current node). So not both strictly smaller, and not both strictly larger. Return 2.

LCA is 2 (correct: 2 is an ancestor of 4, and is itself p).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TreeNode {
5     int val;
6     TreeNode* left;
7     TreeNode* right;
8     TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9 };
10
11 TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q
12 ) {
13     TreeNode* current = root;
14     while (true) {
15         if (p->val < current->val && q->val < current->val) {
16             current = current->left;
17         } else if (p->val > current->val && q->val > current->val) {
18             current = current->right;
19         } else {
20             return current;
21         }
22     }
23 }
24
25 int main() {
26     TreeNode* root = new TreeNode(6);
27     root->left = new TreeNode(2);
28     root->right = new TreeNode(8);
29     root->left->left = new TreeNode(0);
30     root->left->right = new TreeNode(4);
31     root->right->left = new TreeNode(7);
32     root->right->right = new TreeNode(9);
33
34     TreeNode* lca = lowestCommonAncestor(root, root->left, root->left->
35     right);
36     cout << "LCA: " << lca->val << endl;
37     return 0;
38 }

```

Complexity Analysis

Time Complexity. $O(h)$ – a single root-to-LCA walk, never touching any subtree that doesn't lie on the path to the answer. **Space Complexity.** $O(1)$ for the iterative version shown.

Key Takeaway

LCA in a general binary tree needs $O(n)$ because there is no way to know which subtree contains which target without searching both. The BST invariant removes this ambiguity entirely: comparing both targets' values to the current node tells you, with certainty, which

single direction to go (or that you have already arrived), collapsing an $O(n)$ two-subtree search into an $O(h)$ single-path walk.

1.13 Construct a BST from Preorder Traversal

Problem Statement. Given an array representing the preorder traversal of a BST (root, then left subtree, then right subtree, with no duplicate values), reconstruct the BST and return its root.

Pattern Recognition

“Reconstruct a BST from one traversal” works only because the traversal is specifically **preorder** and the tree is specifically a **BST** – the very first element is always the root, and the BST invariant (combined with knowing each value’s relationship to the root) is enough to tell us, for every subsequent element, whether it belongs in the left subtree or the right, without needing a second traversal (like inorder) to disambiguate, the way general binary tree reconstruction usually requires.

Understanding the problem carefully

In a preorder traversal of a BST, the first value is the root. Every value immediately following it that is **less than** the root belongs to the root’s *left* subtree – and because of how preorder works (left subtree fully traversed before any of the right subtree begins), *all* such smaller values appear contiguously, immediately after the root, before any larger value appears. The first value that is **greater** than the root marks the start of the right subtree, and every later value will also be greater than the root.

Build with an Upper Bound, Consuming a Shared Pointer

Maintain a single index pointer into the preorder array, shared across all recursive calls (advanced every time a node is consumed). Define `BUILD(upperBound)`: if the pointer is exhausted, or the next value to consume exceeds *upperBound*, return null (this value belongs to some ancestor’s other subtree, not here). Otherwise, consume the next value as a new node; recursively build its left subtree with *upperBound* tightened to this new node’s value; then recursively build its right subtree with the *original upperBound* unchanged (everything remaining that is still $\leq$ the original bound, but now also $>$ this node’s value, belongs on the right).

Why a single shared pointer and a bound (not an explicit split search) suffices

The key insight is that we never need to explicitly search for “where the left subtree ends and the right subtree begins” – the recursive calls naturally stop consuming values the moment they encounter one that violates the current *upperBound*, which is exactly the boundary between subtrees. Because the shared pointer only ever advances (never resets or searches backward), each value in the preorder array is considered, and consumed, exactly once across the entire construction, regardless of how deeply nested its eventual position in the tree is.

Algorithm

1. Initialize a shared index $i \leftarrow 0$.
2. `BUILD(upperBound)`:

- (a) If $i \geq n$ or $preorder[i] > upperBound$: return null.
- (b) Create a new node with value $preorder[i]$; increment i .
- (c) Recursively set this node's left child to $BUILD(node.val)$.
- (d) Recursively set this node's right child to $BUILD(upperBound)$ (the original bound).
- (e) Return this node.

3. Call $BUILD(+\infty)$ at the top level.

Dry run

Dry Run. Take $preorder = [8, 5, 1, 7, 10, 12]$.

- $BUILD(+\infty)$: consume 8 ($i \rightarrow 1$). Node 8.
- Left: $BUILD(8)$: consume 5 ($5 \leq 8, i \rightarrow 2$). Node 5.
 - Left: $BUILD(5)$: consume 1 ($1 \leq 5, i \rightarrow 3$). Node 1. Left: $BUILD(1)$: next is $7 > 1$, return null. Right: $BUILD(5)$: next is $7 > 5$, return null. Node 1 is a leaf.
 - Right: $BUILD(8)$: consume 7 ($7 \leq 8, i \rightarrow 4$). Node 7. Left: $BUILD(7)$: next is $10 > 7$, return null. Right: $BUILD(8)$: next is $10 > 8$, return null. Node 7 is a leaf.
- Right of 8: $BUILD(+\infty)$: consume 10 ($i \rightarrow 5$). Node 10.
 - Left: $BUILD(10)$: next is $12 > 10$, return null.
 - Right: $BUILD(+\infty)$: consume 12 ($i \rightarrow 6$). Node 12, a leaf.

Resulting tree: root 8, left child 5 (children 1, 7), right child 10 (right child 12).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11  TreeNode* build(vector<int>& preorder, int& i, int upperBound) {
12      if (i >= (int)preorder.size() || preorder[i] > upperBound) {
13          return nullptr;
14      }
15      TreeNode* node = new TreeNode(preorder[i++]);
16      node->left = build(preorder, i, node->val);
17      node->right = build(preorder, i, upperBound);
18      return node;
19  }
20
21  TreeNode* bstFromPreorder(vector<int>& preorder) {
22      int i = 0;
23      return build(preorder, i, INT_MAX);
24  }
25

```

```

26 int main() {
27     vector<int> preorder = {8, 5, 1, 7, 10, 12};
28     TreeNode* root = bstFromPreorder(preorder);
29     cout << "Root: " << root->val << endl;
30     return 0;
31 }

```

Complexity Analysis

Time Complexity. Each element of the preorder array is consumed exactly once, with $O(1)$ work per element, giving

$$O(n).$$

Space Complexity. $O(h)$ for the recursion stack, plus $O(n)$ for the n nodes created.

Key Takeaway

Reconstructing a tree from a single traversal is only possible when that traversal, combined with some known structural invariant (here, the BST property), removes all ambiguity about subtree boundaries. The “pass a bound downward, consume from a shared pointer” technique used here is a direct cousin of Validate BST’s (Section 1.8) bound-passing recursion, repurposed for construction instead of verification.

1.14 Recover BST (Two Nodes Swapped)

Problem Statement. The values of exactly two nodes in a BST were swapped by mistake, breaking the BST invariant. Without changing the tree’s structure, recover the original BST by fixing the values of those two nodes (find them and swap their values back).

Pattern Recognition

“*Exactly two nodes are out of place*” in a problem framed around **sortedness** is the strongest possible signal to invoke Section 1.1’s headline fact directly: a correct BST’s inorder traversal is strictly increasing, so the swapped pair must show up as exactly one or two places where the inorder sequence decreases instead of increases – find those violation points during a single inorder traversal, and the swapped nodes are sitting right there.

Understanding the problem carefully

In a correct BST, scanning the inorder sequence left to right, every value is greater than the one before it. With two values swapped, this property breaks at either one place (if the two swapped nodes happen to be *adjacent* in inorder order) or two places (if they are not adjacent) – and at each violation, the *earlier* of the two compared values is the one that is too large for its position, or the *later* one is too small.

Track the First and Second Violation During One Inorder Pass

Maintain *prev* (the previously visited node) and two not-yet-found pointers, *first* and *second*. During inorder traversal, at each node: if *prev* exists and *prev.val* > *node.val* (a violation): if *first* has not yet been set, set *first* ← *prev*; in either case, set *second* ← *node* (always update to the most recent violation’s second node). Update *prev* ← *node* and continue. After the traversal, swap *first.val* and *second.val*.

Why exactly these two recorded nodes are the swapped pair

If the swap is between adjacent inorder values, exactly one violation occurs, and *first/second* from that single violation are exactly the two swapped nodes. If the swap is between non-adjacent values, exactly *two* violations occur: the first violation's *prev* is the node that ended up holding too large a value (it should be smaller, since the swap displaced a large value into an earlier position), and the second violation's current node is the one holding too small a value (displaced later than it should be). Always keeping *first* fixed at the *first* violation encountered, while letting *second* update to track the *most recent* violation's later node, correctly identifies both endpoints of the original swap regardless of which of these two cases occurred – swapping their values back is enough to restore strict monotonicity, since no third node is ever out of place.

Algorithm

1. Initialize $prev, first, second \leftarrow \text{null}$.
2. Perform an inorder traversal. At each visited node:
 - (a) If $prev \neq \text{null}$ and $prev.val > node.val$:
 - If $first = \text{null}$: $first \leftarrow prev$.
 - $second \leftarrow node$.
 - (b) $prev \leftarrow node$.
3. Swap $first.val$ and $second.val$.

Dry run

Dry Run. A BST whose intended inorder sequence is 1, 2, 3, 4, 5, 6, 7 has nodes holding values 3 and 6 swapped, giving the actual inorder sequence 1, 2, 6, 4, 5, 3, 7.

- Visit 1: $prev = \text{null}$, no check. $prev = 1$.
- Visit 2: $1 > 2$? No. $prev = 2$.
- Visit 6: $2 > 6$? No. $prev = 6$.
- Visit 4: $6 > 4$? Yes – violation. $first = \text{null}$, so $first \leftarrow \text{node holding 6}$. $second \leftarrow \text{node holding 4}$. $prev = 4$.
- Visit 5: $4 > 5$? No. $prev = 5$.
- Visit 3: $5 > 3$? Yes – violation. $first$ already set, unchanged. $second \leftarrow \text{node holding 3}$. $prev = 3$.
- Visit 7: $3 > 7$? No. $prev = 7$.

first is the node currently holding 6; *second* is the node currently holding 3. Swap their values: the node that held 6 now holds 3, and the node that held 3 now holds 6. The inorder sequence becomes 1, 2, 3, 4, 5, 6, 7 – correctly recovered.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 void inorder(TreeNode* node, TreeNode*& prev, TreeNode*& first,
12             TreeNode*& second) {
13     if (!node) return;
14     inorder(node->left, prev, first, second);
15
16     if (prev && prev->val > node->val) {
17         if (!first) first = prev;
18         second = node;
19     }
20     prev = node;
21
22     inorder(node->right, prev, first, second);
23 }
24
25 void recoverTree(TreeNode* root) {
26     TreeNode *prev = nullptr, *first = nullptr, *second = nullptr;
27     inorder(root, prev, first, second);
28     swap(first->val, second->val);
29 }
30
31 int main() {
32     // Built so that an inorder traversal visits 1, 2, 6, 4, 5, 3, 7 --
33     // i.e. the values 3 and 6 have been swapped relative to a correct
34     // BST.
35     TreeNode* root = new TreeNode(4);
36     root->left = new TreeNode(2);
37     root->left->left = new TreeNode(1);
38     root->left->right = new TreeNode(6); // swapped value
39     root->right = new TreeNode(3);      // swapped value
40     root->right->left = new TreeNode(5);
41     root->right->right = new TreeNode(7);
42
43     recoverTree(root);
44
45     TreeNode *p = nullptr, *f = nullptr, *s = nullptr;
46     inorder(root, p, f, s); // verify: no violations should remain
47     cout << (f == nullptr ? "Recovered" : "Still broken") << endl;
48     return 0;
49 }

```

Complexity Analysis

Time Complexity. $O(n)$ – a single inorder traversal visiting every node once. **Space Complexity.** $O(h)$ for the recursion stack. (An advanced variant using Morris traversal

achieves $O(1)$ space, at the cost of temporarily modifying the tree's pointers during the scan – worth knowing exists, though the recursive version here is the one to derive first.)

Key Takeaway

“Exactly k elements are out of place in an otherwise sorted sequence” problems are solved by scanning once and recording violation points, rather than sorting from scratch. Here, $k = 2$ gives exactly the *first/second* bookkeeping shown; the same idea generalizes (with more bookkeeping) to other small, fixed values of k .

1.15 Largest BST in a Binary Tree

Problem Statement. Given the root of a (general, not necessarily search) binary tree, find the size (number of nodes) of the **largest subtree** that is a valid BST.

Pattern Recognition

This closing problem of the chapter is a deliberate capstone: it combines Validate BST's (Section 1.8) range-checking idea with a **bottom-up, tree-DP** style of recursion – each subtree must report enough information upward (is it a valid BST? how big? what are its min and max values?) for its *parent* to decide, in $O(1)$ additional work, whether combining it with its sibling and itself forms an even larger valid BST. The keyword combination “*largest substructure satisfying a property*” within a *general* tree (not already known to be a BST) is the signal for this bottom-up, information-passing recursion style, much like dynamic programming on trees.

Understanding the problem carefully

Section 1.8 validated a BST top-down, passing a valid range *downward*. Here, we do not know in advance which subtrees are valid BSTs, so we instead compute, for *every* subtree, bottom-up: whether it is itself a valid BST, its size if so, and its minimum and maximum values (needed by its parent to check the BST invariant one level up). A node's own subtree is a valid BST exactly when both children report “yes, I am a valid BST,” *and* the node's value is strictly greater than its left child's reported maximum, *and* strictly less than its right child's reported minimum.

Bottom-Up: Each Subtree Reports (isBST, size, min, max)

Recurse postorder (process both children before combining at the current node). For a null node, return ($isBST = \text{true}$, $size = 0$, $min = +\infty$, $max = -\infty$) (a neutral, vacuously-true base case that combines safely with any actual node). At a real node: if both children report $isBST = \text{true}$, and $node.val > leftInfo.max$, and $node.val < rightInfo.min$: this subtree is a valid BST of size $leftInfo.size + rightInfo.size + 1$; update a running global answer with this size, and return (true , $size$, $\min(leftInfo.min, node.val)$, $\max(rightInfo.max, node.val)$). Otherwise, this subtree is not a valid BST: return $isBST = \text{false}$ (its size/min/max no longer matter to any ancestor, since no ancestor can validly combine across an already-invalid subtree).

Why the global answer must be tracked throughout, not just at the root

The largest valid BST is not necessarily the *whole* tree, nor even necessarily a subtree rooted near the root – it could be a valid BST entirely contained within one branch, while everything above

that branch fails the BST invariant. Because the recursion already computes, *at every single node*, whether that node's subtree is a valid BST and how large it is, the correct overall answer is simply the **maximum** of this size over every node visited during the entire traversal – which is why a running global maximum, updated every time a valid BST subtree is found (regardless of whether its ancestors turn out to be valid), is necessary, rather than only inspecting the final result returned at the root.

Algorithm

1. Initialize a global $maxBSTSize \leftarrow 0$.
2. SOLVE($node$):
 - (a) If $node = \text{null}$: return $(\text{true}, 0, +\infty, -\infty)$.
 - (b) $L \leftarrow \text{SOLVE}(node.\text{left})$, $R \leftarrow \text{SOLVE}(node.\text{right})$.
 - (c) If $L.\text{isBST}$ and $R.\text{isBST}$ and $node.val > L.max$ and $node.val < R.min$:
 - $size \leftarrow L.size + R.size + 1$.
 - $maxBSTSize \leftarrow \max(maxBSTSize, size)$.
 - Return $(\text{true}, size, \min(L.min, node.val), \max(R.max, node.val))$.
 - (d) Else: return $(\text{false}, 0, -, -)$.
3. Call $\text{SOLVE}(root)$; return $maxBSTSize$.

Dry run

Dry Run. Take the tree with root 5, left child 2 (left child 1, right child 3, a valid BST of size 3), right child 7 (left child 6, right child 4 – note 4 on the right side violates BST order).

- SOLVE(1): leaf. Returns $(\text{true}, 1, 1, 1)$.
- SOLVE(3): leaf. Returns $(\text{true}, 1, 3, 3)$.
- SOLVE(2): $L = (\text{true}, 1, 1, 1)$, $R = (\text{true}, 1, 3, 3)$. Check: $2 > 1$ and $2 < 3$: valid. Size $= 1 + 1 + 1 = 3$. $maxBSTSize = 3$. Return $(\text{true}, 3, 1, 3)$.
- SOLVE(6): leaf. Returns $(\text{true}, 1, 6, 6)$.
- SOLVE(4): leaf. Returns $(\text{true}, 1, 4, 4)$.
- SOLVE(7): $L = (\text{true}, 1, 6, 6)$ (left child 6), $R = (\text{true}, 1, 4, 4)$ (right child 4). Check: $7 > 6$? Yes. $7 < 4$? No – invalid (right child's minimum 4 is not greater than 7). Return $(\text{false}, -, -, -)$.
- SOLVE(5): $L = (\text{true}, 3, 1, 3)$ (from node 2), $R = (\text{false}, \dots)$ (from node 7). $R.\text{isBST}$ is false, so node 5's subtree is not a valid BST. Return $(\text{false}, -, -, -)$.

The largest valid BST found during the traversal has size 3 (the subtree rooted at node 2), even though the overall tree (rooted at 5) is not a valid BST.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TreeNode {
5      int val;
6      TreeNode* left;
7      TreeNode* right;
8      TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
9  };
10
11 struct Info {
12     bool isBST;
13     int size;
14     int minVal, maxVal;
15 };
16
17 int maxBSTSize = 0;
18
19 Info solve(TreeNode* node) {
20     if (!node) return {true, 0, INT_MAX, INT_MIN};
21
22     Info L = solve(node->left);
23     Info R = solve(node->right);
24
25     if (L.isBST && R.isBST && node->val > L.maxVal && node->val < R.
        minVal) {
26         int size = L.size + R.size + 1;
27         maxBSTSize = max(maxBSTSize, size);
28         return {true, size, min(L.minVal, node->val), max(R.maxVal,
            node->val)};
29     }
30     return {false, 0, 0, 0};
31 }
32
33 int largestBSTSubtree(TreeNode* root) {
34     maxBSTSize = 0;
35     solve(root);
36     return maxBSTSize;
37 }
38
39 int main() {
40     TreeNode* root = new TreeNode(5);
41     root->left = new TreeNode(2);
42     root->left->left = new TreeNode(1);
43     root->left->right = new TreeNode(3);
44     root->right = new TreeNode(7);
45     root->right->left = new TreeNode(6);
46     root->right->right = new TreeNode(4);
47
48     cout << "Largest BST size: " << largestBSTSubtree(root) << endl;
49     return 0;
50 }

```

Complexity Analysis

Time Complexity. Each node is visited exactly once, doing $O(1)$ work combining its children's reported info, giving

$$O(n).$$

This is a substantial improvement over the naive approach of running Validate BST (Section 1.8) independently on every single subtree, which would cost $O(n^2)$. **Space Complexity.** $O(h)$ for the recursion stack.

Key Takeaway

Whenever a problem asks for the largest/best substructure satisfying some recursively-checkable property within a general tree, look for a bottom-up recursion where each subtree reports exactly the information its parent needs to make an $O(1)$ decision – validity, size, and boundary values, in this case. This “return a small summary struct upward, combine in $O(1)$ at each parent” technique is the tree analogue of dynamic programming, and it is the most general and reusable idea in this entire chapter.